UML-Pipeline Application

Detailed Progress and System Design Report

Automated UML Dataset Generation from Natural-Language Requirements
with Multimodal Verification for Software Design

Student: Dipak Yadav (Campus ID 033783670) · Advisor: Dr. Yutong Zhao
M.S. Thesis application — California State University, Long Beach (CECS 698)
Production host: Apple Mac Studio M1 Ultra, 128 GB · Report date: 31 August 2026
Public code: https://github.com/dipak5501/uml-generation-pipeline

1. Purpose and research goal

This report documents the UML-Pipeline application: what the software does, which UML types it produces, where training and test data come from, how the system is designed, which features are implemented, and what has been measured. It also states how each part of the application realizes the thesis method (specification, PlantUML synthesis, render gate, and multimodal verification).

Manual UML modeling is slow and inconsistent. Large language models can draft diagrams from text, but outputs may be syntactically invalid or semantically wrong. This project treats generation and verification as one pipeline so that a requirement or source file becomes a technical specification, then PlantUML, then a rendered image that can be scored and gated for a research dataset of (specification, PlantUML, score) triples.

1.1 How the application implements the thesis

The thesis defines a three-stage method. The application is the runnable realization of that method, plus engineering needed to make generation reliable (validation, repair, grounded PlantUML builders, persistence, and a user interface).

Thesis element	What the application does
Stage 1 — technical specification	LLaMA-class spec model (local Ollama) turns NL or source into JSON (entities, relationships, type-specific fields), then validity + grounding
Stage 2 — PlantUML for class, object, component, package	Paper path: DeepSeek-R1-32B. Local path: LoRA PlantUML + spec-faithful builder so diagrams stay typed and named
Stage 3 — render gate	PlantUML JAR compile and PNG; failed render forces composite score S = 0
Three VLMs + MMMU weights 53.1 / 50.7 / 39.9	Qwen2.5-VL-3B, LLaMA-3.2-Vision-11B, Aya-Vision-8B score each image 0–6 on the four-criterion rubric
Majority acceptance A and composite S	$\tau = 4$ with ≥2 votes; dataset entry requires render_ok, A, and $S \geq 3$
Design-phase UML only	UI and API expose class, object, component, package only
Human evaluation / alignment	Same four criteria in the Human Evaluation page; correlation study not yet run
Public code and dataset tooling	GitHub repository, SQLite artifacts, export filtered by the thesis gate

Thesis method mapped onto the running application.

2. What the application creates: UML diagram types

The product generates four design-phase UML types. Sequence, use-case, deployment, and activity/flowchart diagrams are not offered in the UI. Sequence PlantUML is rejected by UML structure rules (negative control NEG-sequence-unsupported). Flowchart/activity rows may appear in optional training top-up data but are not a thesis product type.

Type	What it shows	Typical PlantUML	When the app uses it
Class	Types, attributes, methods, associations, inheritance, composition/aggregation	class Name { fields; methods() } with --|> *-- o-- -->	Default type; strongest LoRA coverage; structural analysis from source code

Object	Runtime instances, slots, and links between objects	object "name" as alias { slot = value } and links	Instance view of the same domain; historically weaker render rate in bulk NL tests
Component	Deployable/logical components, provided interfaces, dependencies	[Name] as Alias, () "IName", ..> uses	Service/architecture view; local path uses spec-builder until LoRA is proven
Package	Namespaces/modules, nested contents, package dependencies	package Name { ... } and ..> between packages	Module structure; hardest type historically; spec-builder default locally

Table 1. Product UML types. Inputs may be English (or other) requirements or source code (Python, Java, and other languages detected by the code-analysis service).

2.1 Class diagrams

Class diagrams are the primary design artifact. Stage 1 extracts entity names, attributes, and methods from the requirement or from declared types in source. Stage 2 emits PlantUML class blocks and relationships. Inheritance is rendered with --|> only when the spec marks inheritance; otherwise associations, dependencies, composition, or aggregation are used. The fidelity gate replaces placeholder names such as Module1/EntityA with spec names.

2.2 Object diagrams

Object diagrams instantiate types from the specification (for example book1 : Book). They are intended to show a snapshot of collaborating instances, not the type system. In the 1,000-row multilingual NL bulk test, object diagrams were the weakest of the four types among render failures.

2.3 Component diagrams

Component diagrams map domain concepts to components and interfaces (for example [Cart] with ICart). A prior defect produced empty @startuml/@enduml and a blank PNG when LoRA failed and a filter dropped generic names. The application now defaults component generation to the grounded spec-builder, never emits an empty component diagram, and harvests actors and verb objects (Customer, Cart, Product, Order, Payment, and so on). A live e-commerce requirement produced eight named components, compiled, rendered, and passed UML acceptance.

2.4 Package diagrams

Package diagrams group types into packages (core, api, domain-named packages) with dependency arrows. Nesting and dependency semantics are a known failure mode in the literature and in earlier live VLM runs (one package render failure in a 12-run smoke). The application includes a package-failure taxonomy API for analytics.

3. System design

3.1 Deployment view

Production runs on an always-on Apple Mac Studio (M1 Ultra, 128 GB unified memory) under user LaunchAgents. Clients are the Streamlit UI (http://127.0.0.1:8501 and a Cloudflare public URL) and HTTP/CLI callers, including POST /api/agent/command. They call FastAPI on :8000. Orchestration in app/services/orchestration.py runs Stages 1–3. Background jobs use a thread pool. Persistence is SQLite (data/uml_app.db) plus PNG files under data/artifacts/. PlantUML rendering uses a local JDK plus tools/plantuml.jar (optional remote PlantUML HTTP). PlantUML generation uses the MLX LoRA adapter models/uml-plantuml-lora-sourcecode-30k. Fine-tuning is offline (make train-source30k / make finetune), not in the request path.

[Streamlit UI / HTTP clients] → FastAPI routers → orchestration → (providers | PlantUML JAR | SQLite+PNG). Providers: Mock, Ollama (spec + VLMs), MLX LoRA (PlantUML), Hugging Face or OpenAI-compatible (optional), local Aya-Vision weights.

3.2 End-to-end generation flow

1. Intake. The client sends requirement text or source code, a diagram type, and flags (async_mode, skip_vlm). Input is clipped and classified (requirement vs source_code). 2. Stage 1. A chat model (paper: LLaMA 3.2-1B-Instruct; local Ollama llama3.2:1b) writes JSON. ensure_valid_spec merges grounded names from NL or AST-like code analysis. 3. Stage 2. choose_generator selects lora, spec-builder, or llm from adaptation memory. PlantUML is sanitized, validated, and optionally repaired (max 3). 4. Compile and render. plantuml -checkonly then PNG. Render fail ⇒ S = 0. 5. Deterministic acceptance. Syntax, compile, render, UML rules, spec fidelity, semantic/traceability. 6. Stage 3 VLMs

(unless skip_vlm). Three vision models score 0–6 on four criteria. 7. Dual-signal gate. Majority A (τ=4, ≥2 votes) and composite S; dataset flag if render_ok ∧ A ∧ S≥3. 8. Persist. Artifact, scores, repairs, traces; UI gallery and analytics.

3.3 Software modules

Module	Responsibility
app/routers/generate.py	POST /api/generate, /generate/batch, GET /samples
app/routers/artifacts.py	Jobs, artifact CRUD-style reads, PNG, PlantUML text, rescore, repair, library gallery
app/routers/analytics.py	Summary, score distributions, package failures, dataset export, adaptation status, health
app/routers/human_review.py	POST /api/human-review four-criterion rubric
app/services/orchestration.py	run_single_generation: spec → PlantUML → repair → render → VLM → persist
app/services/spec_json.py	JSON validity, NL concept harvest, code-to-spec structure
app/services/plantuml_from_spec.py	Deterministic builders for all four types + fidelity replace
app/services/repair.py / adaptation.py	Category repair strategies; generator/strategy win rates
app/services/acceptance.py / uml_structure.py / traceability.py	Multi-layer accept/reject with failure categories
app/services/scoring.py	MMMU-weighted S, majority A, dataset_entry_accepted
app/providers/	Mock, Ollama, HF, OpenAI-compatible, MLX LoRA, PEFT CUDA, local Aya
app/routers/agent.py	Remote allowlisted commands for the Mac Studio operator API
prompts/*.v1.txt	Versioned Stage-1, Stage-2, VLM, repair, human rubric templates
ui/pages/	Eight Streamlit pages listed in Section 5

Table 2. Principal code modules.

3.4 Provider routing

Stage	Paper model	Local application path
Specification	LLaMA 3.2-1B-Instruct	Ollama llama3.2:1b; mock; optional HF/OpenAI
PlantUML (class)	DeepSeek-R1-Distill-Qwen-32B	MLX LoRA Qwen2.5-0.5B (production: models/uml-plantuml-lora-sourcecode-30k)
PlantUML (object)	Same 32B	LoRA if chosen; fidelity + builder fallback
PlantUML (component, package)	Same 32B	Spec-builder default until LoRA has ≥8 proven samples; skip empty LLM rewrite
VLM Qwen slot	Qwen2.5-VL-3B (w=53.1)	Ollama 0.32 qwen2.5vl:3b on :11435
VLM LLaMA slot	LLaMA-3.2-11B-Vision (w=50.7)	Ollama 0.24 llama3.2-vision:11b on :11434
VLM Aya slot	Aya-Vision-8B (w=39.9)	VLM_AYA_BACKEND=local Hugging Face weights; optional llava stand-in or vLLM

Table 3. Model routing. Dual Ollama is required because 0.32 cannot load llama3.2-vision and 0.24 cannot run qwen2.5vl. scripts/ensure_ollama_dual.sh starts both.

3.5 Scoring formulas (as coded)

Render gate: if PNG rendering fails, S = 0. Otherwise S = Σ(w_j · s_j) / Σ(w_j) over available numeric scores, with w = (53.1, 50.7, 39.9). Unavailable scorers (None) are skipped. Majority: v_j = 1 if s_j ≥ τ (default 4); A = 1 if Σ v_j ≥ 2. Dataset entry: render_ok ∧ A ∧ S ≥ 3.0. VLM rubric criteria: semantic correctness, structural completeness, syntactic accuracy, overall coherence.

3.6 Acceptance and repair

Independent of VLMs, evaluate_acceptance checks syntax, PlantUML compile, render, UML type rules (for example no sequence as a product type), consistency with the spec, and semantic/traceability to the requirement. Repair strategies include sanitize_syntax, inject_missing, strip_hallucinations, spec_rebuild, and llm_targeted. Empty PlantUML from a strategy is discarded. Adaptation memory stores per-type generator and strategy success rates under

data/adaptation_memory.json.

4. Datasets for training, testing, and evaluation

Data fall into four buckets: (A) open Hugging Face UML corpora for LoRA training, (B) locally generated supplemental NL and code rows, (C) application sample requirements and golden tests, (D) evaluation runs that measure the running pipeline. Training files under data/ are gitignored; scripts rebuild them.

4.1 Training corpora (Hugging Face UMLCode)

scripts/build_training_corpus.py assembles a paper-oriented set of about 8,000 rows across class, object, component, and package (target up to 2,000 per type, topped up with extra class rows if a type is short). Optional --include-flowchart appends activity rows; that is not a product diagram type.

Hugging Face dataset	Forced / allowed type	Role
nguyenvanviet/UMLCode-ClassDiagram-DeepSeek-32B-Reasoning-RAW	class	Primary class PlantUML + requirement/spec pairs
nguyenvanviet/UMLCode_ObjectDiagram_Scored	object	Scored object diagrams
nguyenvanviet/UMLCode_ComponentDiagram_Scored	component	Scored component diagrams
nguyenvanviet/UMLCode_PackageDiagram_Scored	package	Scored package diagrams
nguyenvanviet/UMLCode-DeepSeek-32B-Reasoning-UC-Class-Sequence-Scored	class, object, component, package (filtered)	Top-up; sequence/use-case dropped for product types
nguyenvanviet/UMLCode_Activity_Final (optional)	flowchart/activity	Optional fill only; not in UI
nguyenvanviet/UMLCode_DeploymentDiagram (top-up)	filtered to paper types	Used only if count < 8000

Table 4. Open sources listed in scripts/build_training_corpus.py. A gated class-scored HF set is optional via scripts/download_datasets.py --include-gated (HF_TOKEN + license).

Local artifacts after build: data/training/uml_training_8000.parquet and .jsonl, manifest.json. After supplements: uml_training_supplement_merged.parquet. Fine-tune JSONL: data/finetune/{train,valid,test}.jsonl. Production LoRA on the Mac Studio is models/uml-plantuml-lora-sourcecode-30k (6,000 iters, Java/Python/C source corpus) on mlx-community/Qwen2.5-0.5B-Instruct-4bit. NVIDIA hosts use PEFT via scripts/finetune_plantuml_cuda.py. Only Stage 2 PlantUML uses LoRA.

4.2 Supplemental training and stress-test sets (local)

scripts/build_scenario_code_corpus.py synthesizes two extra 1,000-row sets used both as LoRA supplements and as generate/render stress tests (VLM mocked for throughput):

Set	n	Contents	Files
NL scenarios	1,000	20 domains (ecommerce, hospital, banking, education, logistics, IoT, HR, …) × templates in 19 languages (en, es, fr, de, hi, zh, pt, it, ja, ko, ar, ru, nl, tr, pl, sv, vi, th, id)	data/training/scenarios_1000.jsonl; data/eval/scenarios_1000.jsonl
Source code	1,000	20 languages: Python, Java, JavaScript, TypeScript, Rust, Go, C#, Kotlin, Swift, C++, Ruby, PHP, Scala, Dart, Perl, Lua, R, MATLAB, Haskell, Elixir	data/training/code_langs_1000.jsonl; data/eval/code_langs_1000.jsonl
Merged supplement	2,000	Parquet compatible with prepare_finetune_data	data/training/uml_supplement_2000.parquet

Table 5. Locally generated supplemental corpora.

4.3 Application sample and golden tests

Resource	Size	Use
sample_data/requirements.txt	50 English design requirements (bookstore, hospital, fleet, LMS, …)	UI samples API; acceptance benchmark × 4 types = 200 cases
tests/golden/cases.json	6 labeled cases (class, package, component, object)	Golden regression for names/relationships

NEGATIVE_CASES in eval_acceptance.py	5 must-reject diagrams (empty, missing names, hallucinations, bad syntax, sequence)	True-negative rate of the acceptance stack
pytest suite	Unit/API tests under tests/ (acceptance, spec JSON, PlantUML builder, adaptation, validators, API)	Continuous checks of services, not a 8k dataset pass

Table 6. In-repo test fixtures.

4.4 Evaluation campaigns and measured counts

The following counts are what this application has actually run. They must not be conflated with manuscript human-study figures unless those studies are re-executed.

Campaign	Data source	n (how computed)	What was measured	Result
Golden acceptance	tests/golden/cases.json	6	Generate+compile+render+UML+semantic (no VLM)	6/6 accepted
Benchmark acceptance	requirements.txt × {class,object,component,package}	50 × 4 = 200	Same deterministic stack; reports/acceptance_eval.md	200/200 accepted
Negative controls	Hand-crafted invalid PlantUML	5	Must reject	5/5 rejected; 0 false accepts
Live 3-VLM smoke	Stratified types + multilingual + source-code	12	All scorers up, render, mean S	12/12 scorers; 11/12 render; mean S≈4.93
Mac Studio live generate (31 Aug 2026)	Bookstore class diagram via /api/agent generate, skip_vlm=false	1	Render + three-VLM composite S	render success; S ≈ 5.37
NL bulk generate/render	data/eval/scenarios_1000.jsonl	1,000	Valid UML + PNG; VLM mocked	974/1000 (97.4%)
Code bulk generate/render	data/eval/code_langs_1000.jsonl	1,000	Valid UML + PNG; VLM mocked	1000/1000 (100%)

Table 7. Evaluation inventory. Total distinct evaluation items in these campaigns: 6 + 200 + 5 + 12 + 1,000 + 1,000 = 2,223 pipeline runs of different kinds. The 8,000-row HF set is for training, not a full live 3-VLM pass. The 2,000 bulk rows measure render, not S. Interactive UI generate may skip VLMs; batch/rescore apply the paper gate.

5. Application features

5.1 Streamlit UI

Page	Features
Home / Dashboard	API health, artifact counts, render/score snapshot, navigation into generate and batch
Single Generation	NL or paste source code; select class/object/component/package; async job; skip_vlm for faster interactive diagrams; show spec, PlantUML, PNG, acceptance, S/A when scored
Batch Generation	Many requirements, multiple types, progress, job status; intended path for paper scoring
Generated Diagrams	Paginated gallery of rendered UML, PlantUML text, filters, dataset flags
Human Evaluation	Four scores matching the VLM rubric plus comments; stored on the artifact
Analytics	Distributions of S, render rates by type, package-failure breakdown, adaptation win rates
Settings	Provider mix, LoRA path, VLM backend, health of dual Ollama, adaptation status
System Design	In-app architecture text aligned with this report

Table 8. UI feature map.

5.2 API and jobs

REST under /api: generate (sync or async), batch generate, samples, jobs, artifacts, library, image, plantuml download, rescore, repair, human-review, analytics summary and distributions, package-failures, export/dataset (majority + S≥3 filter), adaptation/status, settings/health. Optional API_ACCESS_TOKEN. Input caps (requirement length, batch size). PlantUML preprocessor directives are stripped from labels to reduce injection into the renderer.

5.3 Generation quality features

- Two input modes: natural-language requirements and source code (structure-first for long files).
- JSON Stage-1 spec with validity metrics and grounded concept merge (Cart, Order, … from prose).
- Chain-of-thought stripping so private reasoning is not shown as PlantUML.
- Spec-faithful builders so class/object/component/package stay typed and named.
- Adaptive generator policy and repair ordered by empirical win rate.
- Render-as-gate, MMMU-weighted ensemble, majority vote, dataset export.
- Human rubric parallel to VLMs for later correlation studies.
- Mock providers so the app runs without GPUs; live Mac Studio path uses Ollama + 30k LoRA + local Aya.
- Remote agent: health, generate, smoke-test, training-status, restart API/UI.
- Colab helper for Qwen2.5-VL-3B scoring only (not Aya-8B or LLaMA-11B on free T4).

5.4 Production Mac Studio (always-on)

The thesis demonstration server is an Apple Mac Studio (Mac13,2, M1 Ultra, 128 GB) at /Users/033783670/Desktop/uml-generation-pipeline-main. User LaunchAgents keep API, UI, dual Ollama, caffeinate, and Cloudflare tunnels running while the account stays logged in. Remote Cursor Cloud Agents do not run on this hardware; they operate the Mac through POST /api/agent/command. NVIDIA CUDA LoRA (make finetune-cuda) is documented for other machines and must not be run on this Mac.

Service	Local	Notes
FastAPI	:8000	Live providers; API_ACCESS_TOKEN on public tunnels
Streamlit UI	:8501	Eight pages; public Cloudflare URL
Ollama 0.24	:11434	llama3.2-vision:11b (and llama3.2:1b spec)
Ollama 0.32	:11435	qwen2.5vl:3b
Aya-Vision-8B	in-process MPS	VLM_AYA_BACKEND=local; ≥64 GB RAM
PlantUML LoRA	MLX	models/uml-plantuml-lora-sourcecode-30k
Remote agent	/api/agent	Allowlisted commands; 2 workers

Table 8b. Production Mac Studio services (measured 31 August 2026).

6. Connection to the thesis and remaining work

Milestone	Application status
1. Pipeline completion	Stages 1–3, JSON spec, majority gate, UI, API, remote agent, 30k MLX LoRA, dual Ollama, local Aya on Mac Studio 128 GB: implemented and live
2. Dataset and evaluation	Training corpus scripts + 8k target; 2k supplement; 200-type acceptance; 2k bulk render; 12 live VLM; longer archived 3-VLM tables still needed
3. Human-alignment study	UI + prompt rubric ready; expert correlation study not executed
4. Thesis manuscript and research paper	Draft chapters exist; university format pass and paper revision remaining

Table 9. Thesis work items versus application status.

6.1 Limitations (application)

- On-device Stage 2 is 0.5B LoRA + spec-builder, not DeepSeek-R1-32B.
- 24 GB Macs must not load Aya on MPS; Mac Studio 128 GB is the paper-exact local Aya path.
- LoRA training data is class-heavy; component/package rely on the grounded builder locally.
- Full three-VLM scoring is ~100–120 s per diagram; interactive skip_vlm does not compute S.
- The 1,000+1,000 bulk numbers mock VLMs; they are render success, not dataset-accepted counts.
- The 8,000 HF rows train LoRA; they are not a completed live scored dataset export from this UI.
- Package/object remain harder than class.
- Human vs S correlation is not yet a finished study.

6.2 Next engineering and writing work

- Freeze a reproducible live 3-VLM + acceptance snapshot for thesis tables.
- Package-diagram failure examples from package-failures analytics.
- Scope or run the human-alignment protocol on a stratified subset.
- Thesis formatting, citation check, and paper revision.

7. How to reproduce the local application

Clone https://github.com/dipak5501/uml-generation-pipeline . Create the venv (make install). Mac Studio live path: MOCK_PROVIDERS=false USE_OLLAMA=true USE_FINETUNED_CODE=true FINETUNED_ADAPTER_PATH=models/uml-plantuml-lora-sourcecode-30k VLM_AYA_BACKEND=local, then LaunchAgents or ./scripts/run_local.sh. UI: http://127.0.0.1:8501 . Health: GET /api/settings/health . Remote: POST /api/agent/command . Tests: make test (MOCK_PROVIDERS=true). NVIDIA only: make finetune-cuda. Java and plantuml.jar are required to render. Regenerate this PDF: python scripts/generate_progress_pdf.py . Thesis PDF: python scripts/generate_thesis_draft.py .

Respectfully submitted,
Dipak Yadav
dipak.yadav01@student.csulb.edu
https://github.com/dipak5501/uml-generation-pipeline